13_28_aprile

Tabella dei frame

Oltre alla tabella delle pagina che abbiamo per ogni processo, abbiamo anche la tabella dei frame, ogni record è formato in questo modo:

- *stato*: occupato/libero
- *author*: quale processo lo occupa

Questa tabella viene consultata ogni volta che viene creato un processo per creare la sua relativa tabella delle pagine.

Progettazione di una tabella delle pagine

Per creare un ottima tabella delle pagine è importante curare i seguenti aspetti:

- *velocità*: la velocità di consultazione
- *dimensione*

Affrontiamo il problema della velocità

Soluzione storica: avere un registro per ogni entry della tabella delle pagine (per ogni processo), questa è una soluzione molto veloce, ma funziona solo se ho una tabella di dimensione ridotta. Inoltre il context switch diventa lento.

Soluzione moderna: tabella interamente residente in memoria RAM con registro **PTBR (Page Table Base Register)** che vive anche esso in RAM - questo registro indica alla MMU quale tabella di paginazione usare in base al processo in esescuzione. In pratica in quel registro troviamo l'indirizzo fisico del primo record della tabella di paginazione. Servono due accessi alla memoria per prelevare il dato (uno al PTBR, uno al dato dopo aver calcolato il suo offset rispetto all'indirizzo PTBR), il context switch però è più veloce.

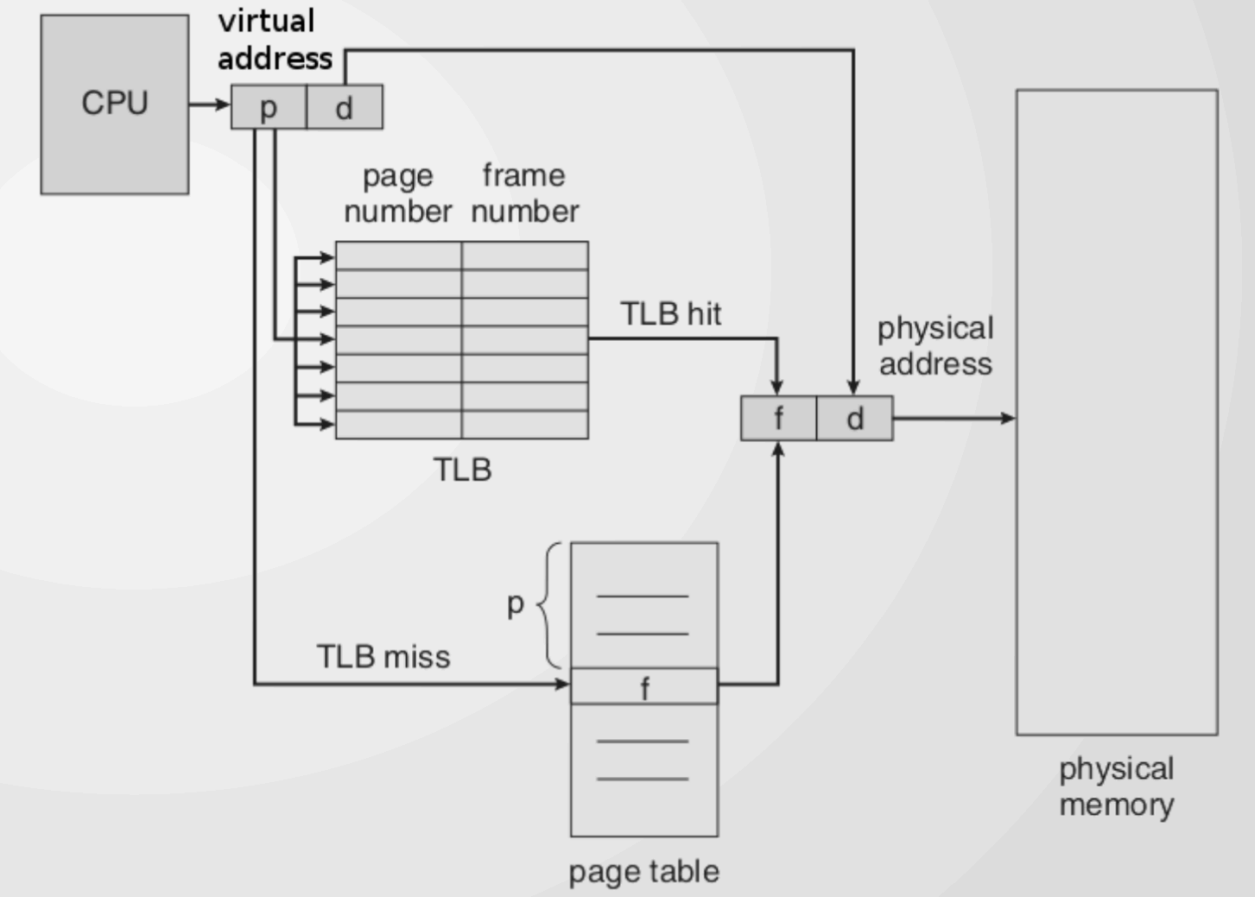
La soluzione moderna è ottima, ma pagare due accessi alla RAM non è buono, cerchiamo una strategia migliore:

Translation Lookaside Buffer (TLB): è una memoria ultra veloce che si frappone tra l'MMU e la RAM (la tabella delle pagine), è praticamente una cache per l'MMU, in questa memoria troviamo gli indirizzi fisici delle pagine virtuali più frequenti richiesti dall'MMU alla RAM.
(è simile quindi ad un prefetch tramite cache per la CPU)

Un programma userà sempre maggiormente una parte ridotta del codice, mettendo in conto questa cosa questo buffer diventa molto utile.

Di seguito varie specifiche:

- viene chiamata anche memoria associativa
- ha un numero ridotto di entry (registri interni) tra 64 e 1024, ognuna di queste ha le seguenti voci:
  - numero di pagina virtuale
  - bit per validità della voce della TLB: indica se la voce è valida (se contiene dati)
  - codice di protezione;
  - dirty bit: indica se la pagina è stata modificata in RAM rispetto alla versione originale in disco
  - numero di frame: questo insieme alla prima voce ci permette di trovare l'indirizzo fisico della pagina specificata
- la ricerca viene fatta parallelizzando su tutte le voci (per questo è molto costosa e piccola, se fosse troppo grande non riusciremmo a parallelizzare)
- essendo molto simile ad una cache abbiamo i concetti di TLB miss e TLB hit
- una volta che una pagina viene liberata dalla RAM il suo rispettivo bit di validità dentro la TLB deve essere modificato
- Ci consente di vincolare alcune pagine alla TLB - è uno strumento che sfrutta il sistema operativo per pagine molto importanti e richieste di frequente
- Quando faccio un context swtich faccio un flush della TLB - la prima ricerca di un processo dopo un flush sarà sempre un miss perché la TLB è vuota.
  Per evitare il flush (è una perdita di tempo assurda se ci sono sempre gli stessi processi) andiamo ad aggiungere un una voce extra detta ASID(address-space identifiers) che identifica il processo dentro la tabella TLB. In pratica adesso la ricerca viene eseguita usando la chiave (page number, ASID).



Riassumendo quando la CPU ha bisogno di un indirizzo se si ha un TLB hit avrò subito a disposizione pagina e frame del processo e quindi gli indirizzi fisici, se si verifica un TLB miss la CPU dovrà reperire la pagina o in RAM o sul disco

Quanto impatta l'utilizzo di una TLB? Di seguito calcoli ci permettono di calcolare il tempo medio (*EAT: Effective access time*)

- Facciamo un **esempio**:
  - tempo di accesso alla memoria = 100 nsec;
  - tempo di accesso alla TLB = 20 nsec;
- **tempo effettivo di accesso** sarà in questo caso:
  - 120 nsec per TLB hit;
  - 220 nsec per TLB miss;
- ipotizziamo un TLB ratio (percentuale di successi) dell'80%;
  - tempo (medio) effettivo di accesso: 0.8 x 120 + 0.2 x 220 = 140 nsec
- in generale:
  - **tempo di accesso alla memoria**: α
  - **tempo di accesso alla TLB**: β
  - **TLB ratio**: ε
    - **EAT** = $\varepsilon(\alpha + \beta) + (1 - \varepsilon)(2\alpha + \beta)$

(nel tempo effettivo di TLB miss è 220 perché 20 + 100 per accedere alla RAM e prendere il valore di PTBR e altri 100 per prendere il dato nella giusta locazione di memoria)

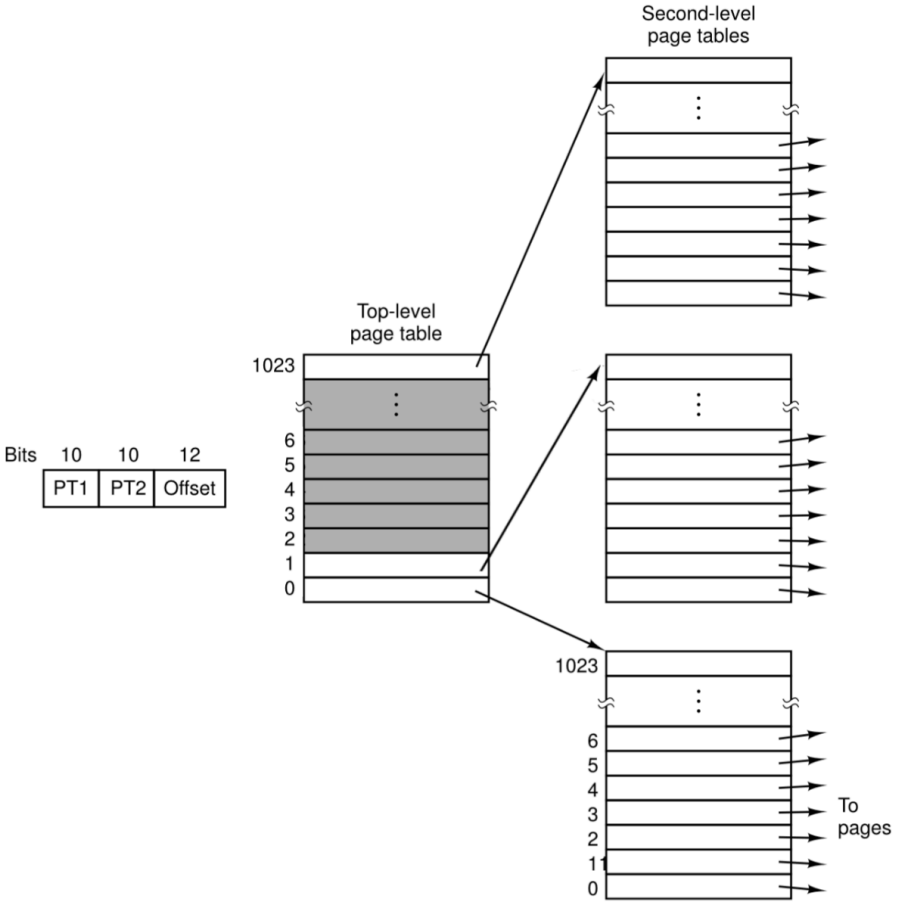
Affrontiamo il problema della dimensione

Nella realtà la tabella delle pagine è enorme e quindi non è mai mono-dimensione

- indirizzi virt. a 32 bit (Pentium): 1 milione di pagine da 4KB;
  - tabella grande circa 4MB (supponendo voci da 32 bit);
  - necessita di memoria contigua;

In pratica dovremmo avere 4MB di ram per ogni pagina (una memoria assurda). Da questo tipo di problema nascono le pagine multi-livello. (sarebbe la gestione multi livello di basi di dati)

Tabelle delle pagine multi-livello: il PTBR punta alla tabella di primo livello, nella tabella di primo livello troviamo dei gruppi di pagine. In pratica il PTRB punta ad uno dei gruppi. Dentro al record di primo livello troviamo il puntatore ad una tabella di secondo livello. La tabella di secondo livello è la solita tabella delle pagine (come la conosciamo).



Vantaggio: consumo molta meno memoria
Svantaggio: facciamo più accessi alla memoria quindi riduciamo la velocità

Cosa faccio per creare l'indirizzo fisico: avendo in input solo il numero di pagina virtuale (che chiameremo n) e l'offset - devo capire a quale gruppo appartiene questa pagina. Per farlo eseguo:

$$\frac{n}{1024}$$

otterrò:

- quoziente: che diventa l'identificatore del gruppo
- resto: l'offset dentro il gruppo specificato
  Usando questi due dati arrivo alla pagina specifica che mi indicherà l'indirizzo fisico.

Per gestire i casi reali con 64 bit il numero di livelli aumenta (almeno 4 o 5 livelli), quindi aumenta il numero di accessi alla RAM, sulla carta questa soluzione non sembra ottima, ma realmente grazie all'utilizzo di diverse TLB lo diventa.

> **Tip**
>
> Se le pagine non hanno valore (il bit di validità è a 0) non ha senso gestirle. La struttura multi-livello viene usata solo per le pagine veramente utili

Tabella delle pagine invertite (non ho capito)

Questa è un'altra soluzione nella gestione delle pagine in memoria, in pratica al posto di avere una tabella delle pagine enorme, creiamo una tabella dei frame ma con le seguenti voci:

- idprocesso (PID)
- pagina virtuale

questo ci permette di sostituire le n tabelle della pagine usate per ogni processo ad una singola tabella usata per qualsiasi processo, in modo da diminuire la dimensione in RAM, dato che sono più compatte.
Come si trova l'indirizzo fisico?

- La CPU genera un indirizzo logico diviso in tre parti: **pid** (chi è il processo), **p** (pagina virtuale), **d** (offset/spiazzamento).
- Si cerca nella tabella invertita la riga che contiene esattamente quella coppia `(pid, p)`.
- L'**indice** `i` di quella riga (cioè la sua posizione nella tabella) corrisponde esattamente al **numero del frame fisico** nella RAM!
- L'indirizzo fisico finale si ottiene unendo l'indice `i` trovato e l'offset `d`.

Riassumendo:

- **Per tradurre gli indirizzi in RAM:** Si usa **UNA SINGOLA** Tabella delle Pagine Invertita per tutto il sistema. È piccola perché ha una riga sola per ogni frame *fisicamente esistente* nella RAM (se hai 4GB di RAM, la tabella mappa solo quei 4GB, indipendentemente da quanti processi hai).
- **Per gestire i Page Fault (quando la pagina non è in RAM):** La tabella invertita mappa *solo* quello che si trova fisicamente in RAM. Se un processo cerca una pagina che è finita sul disco fisso (Swap), la tabella invertita non la troverà. Per questo motivo, il Sistema Operativo deve comunque mantenere salvate (magari su disco) le **tabelle delle pagine classiche (una per processo)** per sapere dove andare a recuperare i dati quando avviene un page fault.